

Flags e Bibliotecas: Aula 04

COMANDO.C

1 O que são flags?

As flags nada mais são que parâmetros que passamos para algum comando, definindo para ele uma maneira específica de agir, por exemplo:

```
ls
```

O comando `ls` nós já conhecíamos (confira nosso segundo PDF da Aula 03), ele lista os arquivos do diretório em que nos encontramos, mas você sabia que no Linux existem diretórios ocultos que não aparecem com o comando `ls` sozinho? Eles começam com um ponto no início, como o famoso `.config/`, presente na maioria das distros Linux. Para conseguir ver esses, é preciso usar a flag `-a`:

```
ls -a
```

Sempre que precisar saber mais detalhes sobre comandos e quais as flags ele possui, utilize o comando `man` para acessar o seu manual. Se acessarmos o manual do comando `ls` temos a seguinte saída:

```
man ls
```

Saída:

LS(1)

User Commands

LS(1)

NAME

ls - list directory contents

SYNOPSIS

ls [OPTION]... [FILE]...

DESCRIPTION

List information about the FILES (the current directory by default). Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

Mandatory arguments to long options are mandatory for short options too.

-a, --all

do not ignore entries starting with .

[...]

LS(1)

User Commands

LS(1)

NAME

ls - list directory contents

SYNOPSIS

ls [OPTION]... [FILE]...

DESCRIPTION

List information about the FILES (the current directory by default). Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

Mandatory arguments to long options are mandatory for short options too.

-a, --all

do not ignore entries starting with .

[...]

Como podemos ver, além de escrever sobre a utilidade do comando, também trouxe a informação que queríamos sobre a flag `-a`.

2 As flags do GCC

O GCC é uma coleção de compiladores para diversas linguagens, uma delas a C, originalmente ele foi projetado para trabalhar em um sistema GNU, um sistema operacional do tipo Unix.

No primeiro PDF das nossas aulas falamos sobre o processo de compilação e como o compilador trabalha para que seu código passe por uma série de otimizações, preservando a sua integridade, e nos PDFs da Aula 03 instalamos o GCC em seu sistema.

Agora vamos explorar suas flags para entendermos seu funcionamento.

Como já vimos na aula, usamos o GCC para compilar nossos programas em C e, por padrão, ele gera um arquivo chamado `a.out`, que executamos assim:

```
./a.out
```

Porém, isso impossibilita compilar códigos diferentes no mesmo diretório sem sobrepor o arquivo executável, pois ambos terão o mesmo nome. Além disso, o nome padrão `a.out` pode não ser muito descritivo para nós, não é mesmo?

A flag `-o`

A flag `-o` nos permite nomear o arquivo de saída. Mas atenção, também podemos mudar sua extensão.

```
gcc main.c -o Programa
```

Se rodarmos um `ls` teremos o seguinte resultado:

```
main.c Programa
```

O arquivo Programa segue sendo um executável e usamos `./Programa` para executá-lo normalmente, porém, se tiver olhos bem atentos perceberá que ele não possui a extensão `.out`. Acontece que em sistemas Linux – como é nosso caso graças ao WLS – isso não muda o comportamento do programa executável, mas em sistemas nativamente Windows isso faz total diferença, é necessário então passar a extensão do arquivo quando chamar: `gcc main.c -o Programa.exe`.

As flag de Warning

Warnings são mensagens de diagnóstico geradas pelo compilador, servem para alertar programadores sobre possíveis problemas no seu código, o que não impede a compilação, mas aponta para um comportamento inesperado ou erros em potencial. Especialmente em programas mais complexos, como aqueles que trabalham com alocação dinâmica de memória e estruturas de dados mais abstratas, o compilador terá muito mais a nos dizer sobre a nossa implementação.

Dentro os mais diversos tipos de flags desse grupo, vamos falar brevemente sobre as mais prováveis de usarmos no futuro. São elas, as flags:

-Wall

-Wall

Mostra os avisos mais comuns e que costumam gerar bugs no seu código, como variáveis não inicializadas.

-Wextra

-Wextra

Mostra alertas adicionais, não contemplados no -Wall, como parâmetros de funções não utilizados. Faz uma limpeza nas redundâncias do código e o deixa melhor estruturado.

-Wpedantic

-Wpedantic

Sugere Warnings de acordo com as normas e padrões da ISO C/C++, isso permite uma melhor portabilidade em outros sistemas e compiladores.

-Werror

-Werror

Transforma qualquer Warning emitido pelo compilador em um erro fatal de compilação. Impede a geração do executável até que absolutamente todos os problemas sejam corrigidos.

A flag -O

Como sabemos, o compilador otimiza nosso código, mas podemos fazer o processo de otimização novamente no código Assembly, a fim de passar um pente fino na nossa aplicação, isso pode ser muito útil dependendo do caso. Dentro das flags de otimização, podemos escolher o nível desse processo, quanto maior mais demorada a compilação, ele vai de -O0 (a opção default quando executamos `gcc main.c`) até -O3.

As flags podem, e devem, ser usadas em conjunto umas com outras. Podemos usar as flags -Wall e -Werror:

```
gcc -Wall -Werror main.c
```

3 Escape Sequences

Quando nós começarmos a escrever programas em C, vamos perceber que certos caracteres são usados na sintaxe da linguagem, o que implica que se quisermos usá-los precisamos fazer isso de outra forma. Outra coisa, é que também podemos formatar as saídas no nosso terminal, com a função `printf()`. Isso tudo é possível graças aos escape sequences disponíveis em C.

Eles são usados para representar certos caracteres especiais, abaixo temos uma lista de todos eles:

```
\'
```

Escreve as aspas simples.

```
\"
```

Escreve as aspas duplas.

```
\\
```

Escreve a barra invertida.

```
\a
```

Emite um alarme sonoro (costuma não funcionar).

```
\b
```

Backspace, ele apaga o último caractere digitado, pode ser usado para criar *spinners* giratórios.

```
\f
```

Form feed, antigamente os terminais literalmente imprimiam as suas respostas, e esse comando era utilizado como um sinal ao hardware da impressora para que ela entendesse que deveria ejetar a página atual. Hoje em dia os terminais não são mais físicos, mas esse escape sequence pode ser implementado em uma máquina de cartão de crédito, imprimindo a nota fiscal.

```
\n
```

Line feed, é equivalente a pressionar **Enter**, ou seja, faz um quebra de linha no terminal.

`\r`

Carriage return, ele reescreve a mensagem no terminal, substituindo a antiga por uma nova. É muito usado para atualizar uma linha durante um processo longo, como em uma barra de carregamento.

`\t`

Tabulação horizontal, equivalente a pressionar **Tab**, ou seja, ele “empurra” o texto para a direita, horizontalmente.

`\v`

Tabulação vertical, herda o mesmo comportamento do `\t`, com o diferencial de que escreve em uma nova linha e empurra consideravelmente o texto para a direita, gera um texto em “escadinha”.

4 Bibliotecas em C

Nós aprendemos no PDF anterior que as funções podem ser declaradas, definidas e chamadas na linguagem C, mas tomemos de exemplo a função `printf()`, nós a chamamos na nossa função `main()`, porém, não a declaramos e nem a definimos, então de onde ela vêm?

A resposta é simples, ela vem de uma biblioteca chamada **stdio**.

De maneira resumida, bibliotecas em C são arquivos contendo uma série de funções para um propósito determinado, por exemplo, a biblioteca **stdio** é um acrônimo para *standard input/output*, traduzindo, *Entrada e Saída primários*, essa biblioteca oferece uma série de funções para trabalhar com entrada, saída e manipulação de arquivos. Neste PDF não entraremos em muitos detalhes em como as bibliotecas funcionam, mas trazemos uma lista das principais bibliotecas em C:

Biblioteca	Uso
stdio	Funções de entrada, saída e manipulação de arquivos
stdlib	Funções comumente usadas (Ex: alocação de memória e conversão <code>string</code> para <code>int</code>)
string	Funções para trabalhar com tipo <code>string</code>
math	Funções para trabalhar com cálculos matemáticos mais complexos
ctype	Funções para realizar transformações e classificação de caracteres
time	Funções para trabalhar com data, hora e medição de tempo de execução de programas

5 Exercícios

1. Vamos exercitar o uso das flags de warning do GCC, nesse exercício você deve criar os programas em C e compilar usando o comando `gcc` junto das flags escritas no enunciado. Ao fazer como se pede, você deve estudar o comportamento desses comandos, ou seja, escrever com suas palavras o que entendeu da execução. Os programas possuem erros propositais, para que as flags identifiquem e respondam no terminal.
 - a. Estude o comportamento da flag `-Wall`

```
int calcular_dobro() {  
  
}  
  
int main() {  
    calcular_dobro();  
    return 0;  
}
```

Para esse exercício rode o comando `gcc -Wall ex1_wall.c -o programa1`.

- b. Estude o comportamento da flag `-Wextra`

```
int somar(int numero) {  
    return 5;  
}  
  
int main() {  
    somar(10);  
    return 0;  
}
```

Para esse exercício, você deve rodar 2 comandos e descrever a diferença entre eles:

```
gcc -Wall ex2_wextra.c -o programa2_sem_wextra
```

e

```
gcc -Wall -Wextra ex2_wextra.c -o programa2_com_wextra
```

- c. Estude o comportamento da flag `-Wpedantic`

```
int main() {
    return 0;
};
```

Para esse exercício rode o comando `gcc -Wall -Wpedantic ex3_wpedantic.c -o programa3`.

d. Estude o comportamento da flag `-Werror`

```
int calcular_dobro() {

}

int main() {
    calcular_dobro();
    return 0;
}
```

Note que estamos usando o mesmo código da letra A, isso diz respeito ao comportamento da tag `-Werror`.

Para esse exercício, você deve rodar 2 comandos e descrever a diferença entre eles:

```
gcc -Wall ex4_werror.c -o programa4-sem_werror
```

e

```
gcc -Wall -Werror ex4_werror.c -o programa4_com_werror
```

2. Nesse exercício devemos usar os escape sequence corretos para imprimir o resultado desejado no terminal.

a. Faça com que o seguinte texto seja impresso no terminal, respeitando o espaçamento:

```
--- MENU PRINCIPAL ---
1. Novo Jogo
2. Carregar Jogo
3. Opcoes
4. Sair
Escolha uma opcao:
```

b. Faça com que o seguinte texto seja impresso no terminal, respeitando o espaçamento:

ALUNO	DISCIPLINA	NOTA
João	Matemática	9.5
Maria	História	10.0
Carlos	Física	7.2

c. Faça com que o seguinte texto seja impresso no terminal, respeitando o espaçamento:

```
Linguagem
  C
    é
      Incrível!
```

d. Dessa vez faremos diferente, execute esse código para entender o funcionamento dos escapes sequences `\b` e `\r`:

```

#include <stdio.h>

#ifdef _WIN32
    #include <windows.h>
#else
    #include <unistd.h>
#endif

void pausa(int milissegundos) {
#ifdef _WIN32
    Sleep(milissegundos);
#else
    usleep(milissegundos * 1000);
#endif
}

int main() {
    printf("--- INICIANDO SISTEMA ---\n\n");

    printf("Conectando ao servidor: ");

    char spinner[] = {'|', '/', '-', '\\'};

    for (int i = 0; i < 15; i++) {
        printf("%c", spinner[i % 4]);
        fflush(stdout);

        pausa(150);

        printf("\b");
    }
    printf("CONECTADO!\n\n");

    for (int i = 0; i ≤ 100; i += 5) {
        printf("\rBaixando arquivos... [%d%%]", i);
        fflush(stdout);

        pausa(100);
    }

    printf("\n\nDownload concluido com sucesso!\n");

    return 0;
}

```

6 Gabarito

1. No primeiro exercício temos que dissertar sobre o comportamento das flags de Warning.

a. Na letra A, executamos o comando de compilação e temos:

```
ex1_wall.c: In function 'calcular_dobro':
ex1_wall.c:3:1: warning: control reaches end of non-void function
[-Wreturn-type]
   3 | }
     | ^
```

Graças a flag `-Wall`, fomos alertados de que nossa função `calcular_dobro()` não está retornando nenhum valor, isso potencialmente poderia causar *bugs* no nosso programa, pois esses são os tipos de avisos: “Não vou impedir a compilação, mas isso aqui não está certo e pode ser que gere problemas indesejados depois, siga por sua conta em risco”.

b. Na letra B, temos duas opções de comandos para usar, a primeira não gera nenhum aviso no terminal, já com a segunda temos:

```
ex2_wextra.c: In function 'somar':
ex2_wextra.c:1:15: warning: unused parameter 'numero' [-Wunused-
parameter]
   1 | int somar(int numero) {
     |             ~~~~~^~~~~~
```

Quando executamos esse código com somente a flag `-Wall`, não somos avisados sobre nada, pois essa flag não abrange todos os erros possíveis ou implementações indesejadas. Quando chamadas as duas flags juntas, elas acusam que o parâmetro `numero` não está sendo usado no código da função, de fato isso não iria gerar nenhum erro ou bug, mas certamente deixa nosso código com uma cara bem mais amadora e desatenta, por isso o aviso é gerado.

c. Na letra C, executamos o comando de compilação e temos:

```
ex3_wpedantic.c:3:2: warning: ISO C does not allow extra ';'
outside of a function [-Wpedantic]
   3 | };
     | ^
```

Note que aqui estamos usando o comando com `-Wall` incluso, como já discutimos, as flags possuem um perímetro de ação e é uma boa prática usarmos elas em conjunto.

A flag `-Wpedantic` trás avisos para qualquer incompatibilidade ou desacordos entre seu código e a ISO oficial da linguagem.

d. Na letra D, temos duas opções de comandos para usar, a primeira gera a seguinte saída:

```
ex4_werror.c: In function 'calcular_dobro':
ex4_werror.c:3:1: warning: control reaches end of non-void
function [-Wreturn-type]
    3 | }
      | ^
```

E a segunda:

```
ex4_werror.c: In function 'calcular_dobro':
ex4_werror.c:3:1: error: control reaches end of non-void function
[-Werror=return-type]
    3 | }
      | ^
cc1: all warnings being treated as errors
```

Aqui é muito simples, nós inclusive reutilizamos o código da letra A para demonstrar. A flag `-Werror` transforma todos os warnings em erros, isso impede que o compilador gere o executável do seu código, a flag `-Werror` é perfeita para desenvolver programas profissionais, e em conjunto com as demais flags, garantindo a qualidade e portabilidade do nosso código.

2. No segundo exercício temos que imprimir textos respeitando o espaçamento, o que envolve usarmos os escapes sequences corretos. Abaixo uma lista com os códigos sugeridos, lembrando que a letra D é apenas demonstrativa.

a. Código sugerido:

```
#include <stdio.h>

int main(){
    printf("\n--- MENU PRINCIPAL ---\n");
    printf("1. Novo Jogo\n");
    printf("2. Carregar Jogo\n");
    printf("3. Opcoes\n");
    printf("4. Sair\n");
    printf("Escolha uma opcao:");

    return 0;
}
```

Notou o \n no começo de --- MENU . . .? Pode ser que se você tirasse esse escape sequence, a linha impressa ficaria “grudada” com seu nome de usuário do sistema:

```
comando@debian:~$--- MENU PRINCIPAL ---
1. Novo Jogo
2. Carregar Jogo
3. Opcoes
4. Sair
Escolha uma opcao:
```

b. Código sugerido:

```
#include <stdio.h>

int main()
{
    printf("NOME\tDISCIPLINA\tNOTA\n");

    printf("João\tMatemática\t9.5\n");
    printf("Maria\tHistória\t10.0\n");
    printf("Carlos\tFísica\t\t7.2\n");

    return 0;
}
```

O caractere de tabulação \t não insere uma quantidade fixa de espaços, mas desloca o cursor para posições absolutas no terminal, conhecidas como paradas de tabulação (tab stops). Configuradas nativamente a cada 8 caracteres (colunas 8, 16, 24, etc.), normalmente. O terminal identifica a posição atual do cursor e o avança diretamente

para a próxima parada disponível, gerando um espaçamento dinâmico que varia conforme o comprimento do texto que o antecede, por conta disso colocamos um \t extra no último printf(), logo após a palavra Física.

c. Código sugerido:

```
#include <stdio.h>

int main()
{
    printf("A Linguagem\n");
    printf("\tC\n");
    printf("\t\té\n");
    printf("\t\t\tIncrível!");
}
```